

TUGAS MACHINE LEARNING

Introduction to Machine Learning with Python

Chapter 3: Unsupervised Learning and Preprocessing



Disusun Oleh:

Nama : Aisyah Azzahrah
NPM : G1A024033

Kelas:

Informatika (A)

Dosen Pengampu:

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU
2026**

PEMBAHASAN

Unsupervised learning adalah kelompok algoritma machine learning yang bekerja pada data tanpa label. Model harus menemukan sendiri struktur atau pola yang tersembunyi di dalam data. Ada dua tantangan utama pada pendekatan ini. Pertama, hasilnya sulit dievaluasi secara objektif karena tidak ada label pembanding seperti pada supervised learning, sehingga penilaian sering dilakukan secara visual atau berdasarkan pengetahuan domain. Kedua, unsupervised learning sering digunakan sebagai langkah awal sebelum tahap pemodelan lain, misalnya untuk melakukan scaling data sebelum masuk ke model supervised, atau melakukan reduksi dimensi sebelum data divisualisasikan.

Secara umum, Chapter 3 membahas dua kelompok teknik besar, yaitu transformasi data tak berlabel (preprocessing dan reduksi dimensi) serta clustering (pengelompokan data menjadi beberapa grup berdasarkan kemiripannya).

1. Preprocessing dan Scaling

Banyak algoritma machine learning, terutama yang berbasis jarak seperti SVM dan k-NN, atau yang berbasis gradient seperti neural network, sangat sensitif terhadap skala fitur. Apabila satu fitur memiliki rentang nilai 0 sampai 1 sedangkan fitur lain memiliki rentang 0 sampai 1000, model dapat menjadi bias terhadap fitur dengan skala yang lebih besar. Oleh karena itu, data perlu discaling terlebih dahulu sebelum digunakan. Scikit-learn menyediakan beberapa jenis scaler, di antaranya:

- StandardScaler — membuat setiap fitur memiliki mean 0 dan varians 1, cocok digunakan untuk kebanyakan kasus umum.
- MinMaxScaler — menekan seluruh nilai fitur ke dalam rentang tertentu, umumnya 0 sampai 1.
- RobustScaler — mirip dengan StandardScaler, namun menggunakan median dan kuartil sehingga lebih tahan terhadap outlier.
- Normalizer — menskalakan setiap baris data (bukan kolom) sehingga panjang vektornya menjadi 1, sering digunakan ketika arah data lebih penting dibandingkan besarannya.

Berikut contoh penerapan MinMaxScaler pada dataset kanker payudara (breast cancer dataset):

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

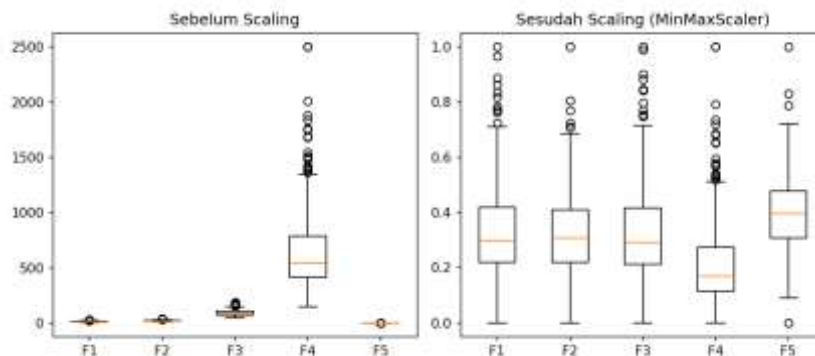
cancer = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, random_state=1
```

)

```
scaler = MinMaxScaler()  
scaler.fit(X_train) # scaler hanya belajar dari data train
```

```
X_train_scaled = scaler.transform(X_train)  
X_test_scaled = scaler.transform(X_test)
```

```
print("Sebelum scaling, nilai minimum fitur:")  
print(X_train.min(axis=0)[:5])  
print("Sesudah scaling, nilai minimum fitur:")  
print(X_train_scaled.min(axis=0)[:5])
```



Gambar 1.1 Distribusi 5 fitur pertama sebelum dan sesudah MinMaxScaler

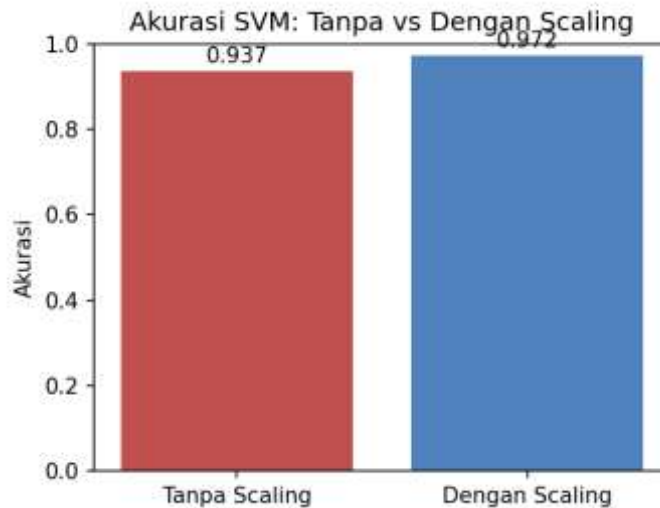
Terlihat pada Gambar 1.1 bahwa sebelum scaling, rentang nilai antar fitur sangat berbeda-beda, sedangkan sesudah scaling seluruh fitur berada pada rentang 0 sampai 1 yang seragam.

Satu hal penting yang harus diperhatikan adalah scaler harus di-fit hanya menggunakan data training, kemudian digunakan untuk mentransformasi data test. Apabila scaler di-fit secara terpisah untuk data train dan data test, hubungan antara kedua data tersebut akan rusak sehingga hasil evaluasi model menjadi tidak dapat dipercaya.

Pengaruh Scaling terhadap Performa Model

Untuk melihat pengaruh scaling secara nyata, performa algoritma SVM dibandingkan antara data yang belum discaling dengan data yang sudah discaling:

```
from sklearn.svm import SVC  
  
svm = SVC(C=100)  
svm.fit(X_train, y_train)  
print("Akurasi tanpa scaling: {:.3f}".format(svm.score(X_test, y_test)))  
  
svm_scaled = SVC(C=100)  
svm_scaled.fit(X_train_scaled, y_train)  
print("Akurasi dengan scaling: {:.3f}".format(svm_scaled.score(X_test_scaled, y_test)))
```



Gambar 1.2 Perbandingan akurasi SVM tanpa dan dengan scaling

Hasil percobaan pada Gambar 1.2 menunjukkan peningkatan akurasi dari 0,937 menjadi 0,972 setelah data discaling. Hal ini menunjukkan bahwa proses preprocessing bukan sekadar langkah formalitas, melainkan dapat sangat menentukan apakah suatu model bekerja dengan baik atau tidak.

2. Reduksi Dimensi

Data yang memiliki terlalu banyak fitur seringkali sulit untuk divisualisasikan maupun diproses secara efisien. Untuk mengatasi hal ini, digunakan teknik-teknik reduksi dimensi seperti PCA, NMF, dan t-SNE.

2.1 Principal Component Analysis (PCA)

PCA bekerja dengan mencari arah (disebut principal component) di mana data memiliki variasi paling besar, kemudian memutar data ke arah tersebut. Fitur baru hasil PCA tidak memiliki makna langsung seperti fitur aslinya, namun sangat berguna untuk kompresi informasi maupun visualisasi.

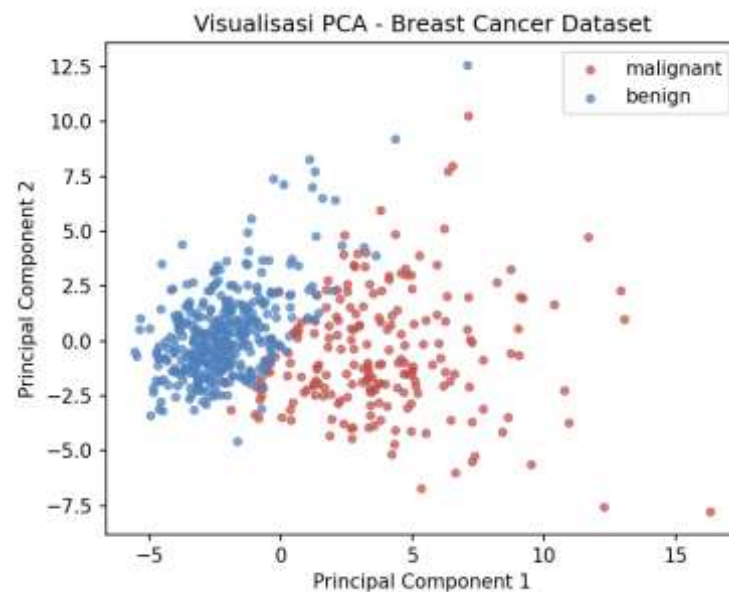
```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

```
scaler = StandardScaler()
scaler.fit(cancer.data)
X_scaled = scaler.transform(cancer.data)
```

```
pca = PCA(n_components=2)
pca.fit(X_scaled)
X_pca = pca.transform(X_scaled)
```

```
print("Bentuk data asli:", X_scaled.shape)
```

```
print("Bentuk data setelah PCA:", X_pca.shape)
```



Gambar 2.1 Visualisasi 2 komponen utama hasil PCA pada Breast Cancer Dataset

Pada Gambar 2.1 terlihat bahwa meskipun PCA tidak diberikan informasi label sama sekali, dua komponen utama yang dihasilkan sudah cukup mampu memisahkan kelompok data malignant dan benign secara visual.

Data yang awalnya memiliki 30 fitur dapat direduksi menjadi 2 komponen utama, yang selanjutnya dapat divisualisasikan dalam bentuk scatter plot dua dimensi. Perlu diperhatikan bahwa PCA sangat sensitif terhadap skala fitur, sehingga data hampir selalu perlu discaling terlebih dahulu sebelum diproses dengan PCA, karena jika tidak, fitur dengan skala besar akan mendominasi arah komponen utama yang dihasilkan.

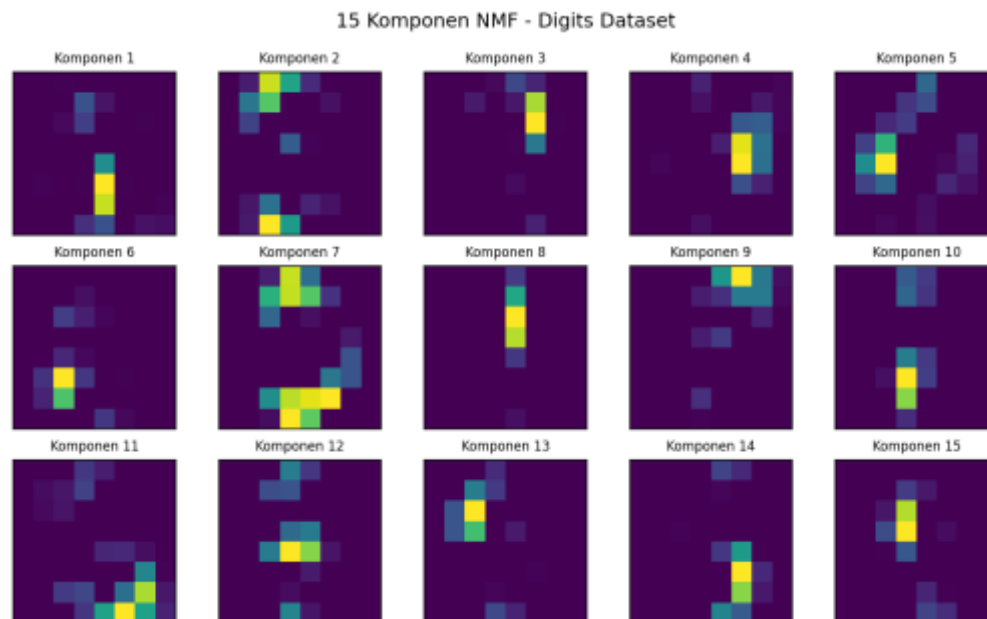
2.2 Non-Negative Matrix Factorization (NMF)

NMF memiliki kemiripan dengan PCA, namun dengan batasan tambahan, yaitu komponen dan koefisiennya harus bernilai non-negatif. Karakteristik ini membuat NMF cocok digunakan pada data yang secara alami tidak bernilai negatif, misalnya intensitas piksel gambar atau frekuensi kata pada data teks. Hasil dari NMF cenderung lebih mudah diinterpretasikan karena komponennya merepresentasikan bagian-bagian yang muncul bersama, bukan kombinasi penjumlahan dan pengurangan seperti pada PCA.

```
from sklearn.decomposition import NMF
from sklearn.datasets import load_digits

digits = load_digits()
nmf = NMF(n_components=15, random_state=0, max_iter=1000)
nmf.fit(digits.data)
```

```
# komponen bisa divisualisasikan sebagai gambar 8x8 piksel  
for component in nmf.components_:  
    component.reshape(8, 8)
```



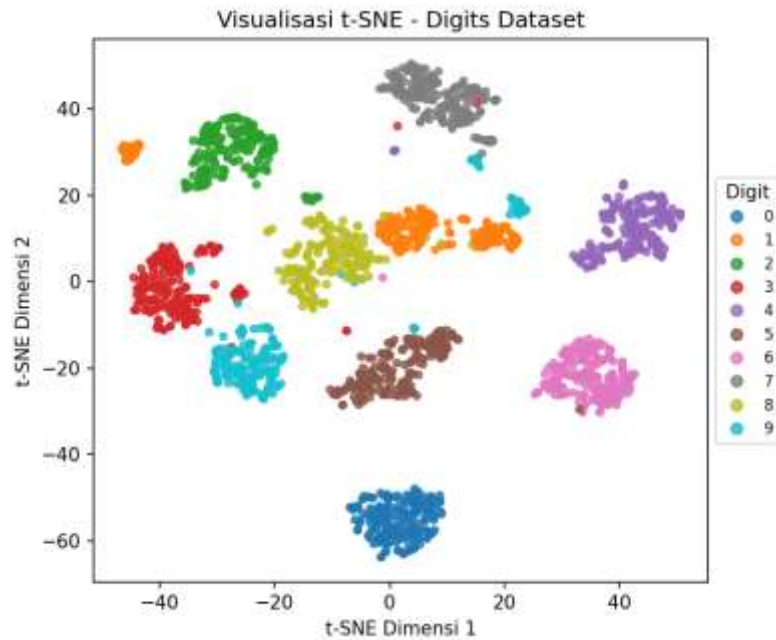
Gambar 2.2 15 komponen hasil NMF pada Digits Dataset

Pada Gambar 2.2, setiap komponen NMF merepresentasikan pola potongan angka (misalnya lengkungan atau garis tertentu) yang muncul berulang pada berbagai digit, dan karena semua nilainya non-negatif, komponen-komponen ini bisa langsung diinterpretasikan sebagai “bagian penyusun” gambar digit.

2.3 t-SNE (Manifold Learning)

Berbeda dengan PCA dan NMF yang bersifat linear, t-SNE lebih difokuskan untuk keperluan visualisasi dan mampu menangkap struktur data yang bersifat non-linear. Prinsip dasarnya adalah titik-titik yang saling berdekatan pada data asli akan tetap dijaga berdekatan pada hasil visualisasi dua dimensi, meskipun hubungan jarak secara global tidak selalu dipertahankan secara persis.

```
from sklearn.datasets import load_digits  
from sklearn.manifold import TSNE  
  
digits = load_digits()  
tsne = TSNE(random_state=42)  
digits_tsne = tsne.fit_transform(digits.data)
```



Gambar 2.3 Visualisasi t-SNE pada Digits Dataset

Gambar 2.3 menunjukkan bahwa data digit 0 sampai 9 membentuk kelompok-kelompok yang terpisah cukup jelas, padahal algoritma t-SNE tidak pernah diberi tahu label digit-digit tersebut.

Hasil t-SNE umumnya digunakan untuk melihat apakah kelas-kelas pada data, misalnya digit 0 sampai 9, membentuk kelompok yang saling terpisah dengan jelas, meskipun algoritma tidak pernah diberikan informasi label sama sekali. Berbeda dengan PCA, t-SNE tidak memiliki method transform yang terpisah, sehingga harus dilakukan fit ulang setiap kali digunakan pada data baru.

3. Clustering

Clustering merupakan tugas untuk membagi data menjadi beberapa kelompok (cluster), sedemikian rupa sehingga data dalam satu cluster memiliki kemiripan satu sama lain, dan berbeda dengan data pada cluster yang lain.

3.1 k-Means

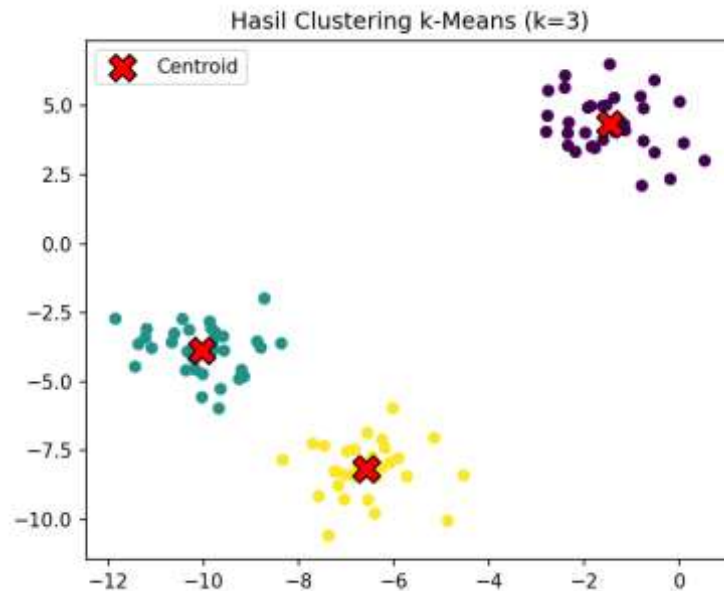
k-Means merupakan algoritma clustering yang paling umum digunakan. Cara kerjanya adalah dengan menentukan jumlah cluster (k) terlebih dahulu, kemudian algoritma akan secara iteratif mencari titik pusat (centroid) untuk masing-masing cluster dan menetapkan setiap titik data ke centroid terdekat, hingga posisi centroid tidak lagi berubah.

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
```

```
X, y = make_blobs(random_state=1)

kmeans = KMeans(n_clusters=3, random_state=0, n_init=10)
kmeans.fit(X)

print("Label cluster untuk tiap titik:")
print(kmeans.labels_)
```



Gambar 3.1 Hasil clustering k-Means ($k=3$) beserta titik centroid

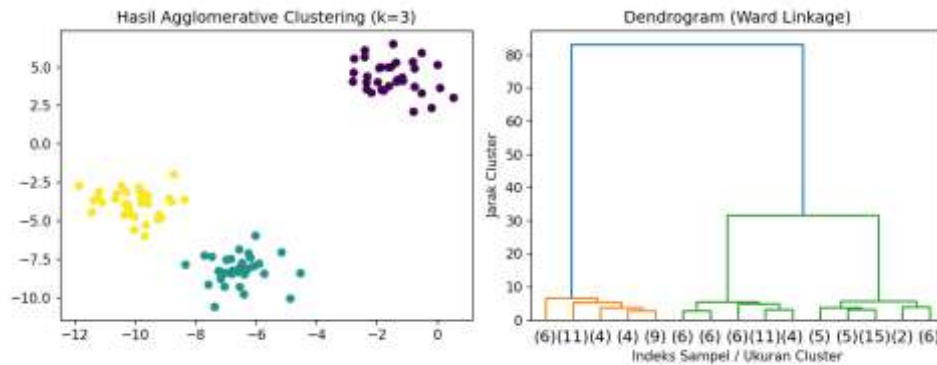
Kelemahan utama k-Means adalah jumlah cluster (k) harus ditentukan terlebih dahulu, padahal jumlah cluster yang sebenarnya seringkali tidak diketahui. Selain itu, k-Means berasumsi bahwa bentuk cluster adalah bulat (sferis) dengan ukuran yang relatif mirip, sehingga jika bentuk cluster tidak beraturan atau memanjang, hasil clustering yang diperoleh bisa menjadi kurang akurat.

3.2 Agglomerative Clustering

Agglomerative Clustering merupakan pendekatan hierarkis, di mana pada awalnya setiap titik data dianggap sebagai cluster tersendiri, kemudian dua cluster yang paling mirip akan digabungkan secara berulang hingga tersisa jumlah cluster yang diinginkan.

```
from sklearn.cluster import AgglomerativeClustering

agg = AgglomerativeClustering(n_clusters=3)
assignment = agg.fit_predict(X)
```



Gambar 3.2 Hasil Agglomerative Clustering dan dendrogram (Ward linkage)

Proses penggabungan cluster pada algoritma ini dapat divisualisasikan dalam bentuk dendrogram, yang berguna untuk melihat pada tingkat mana cluster-cluster mulai terpisah satu sama lain.

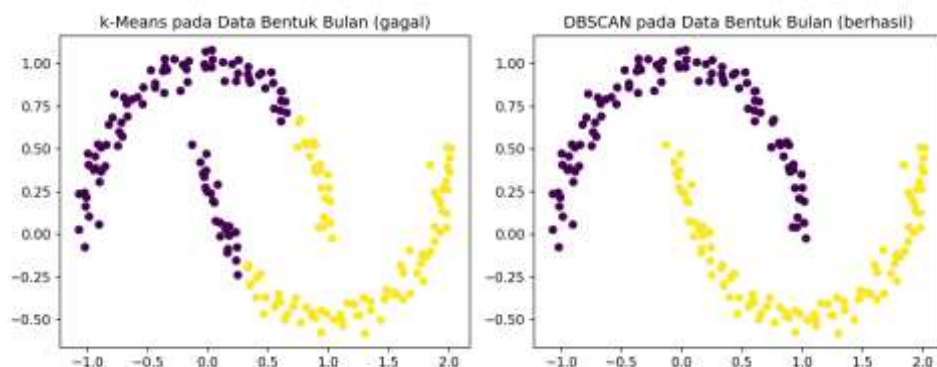
3.3 DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) menggunakan pendekatan yang berbeda dari k-Means maupun Agglomerative Clustering. Algoritma ini tidak memerlukan jumlah cluster ditentukan sejak awal, serta dapat mendeteksi titik data yang dianggap sebagai noise (tidak termasuk ke dalam cluster mana pun). DBSCAN bekerja berdasarkan kepadatan titik data, yaitu apabila di sekitar suatu titik dalam radius tertentu (eps) terdapat minimal sejumlah titik lain (min_samples), maka area tersebut dianggap padat dan menjadi bagian dari suatu cluster.

```
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons
```

```
X_moons, y_moons = make_moons(n_samples=200, noise=0.05, random_state=0)
```

```
dbscan = DBSCAN(eps=0.3, min_samples=5)
clusters = dbscan.fit_predict(X_moons)
```



Gambar 2.8 Perbandingan hasil k-Means (gagal) dan DBSCAN (berhasil) pada data berbentuk bulan

DBSCAN sangat unggul pada kasus seperti dataset `make_moons` di atas, yaitu ketika bentuk cluster melengkung dan tidak bulat, yang biasanya membuat k-Means gagal total namun dapat ditangani dengan baik oleh DBSCAN. Titik data yang diberi label -1 pada hasil DBSCAN menunjukkan bahwa titik tersebut dianggap sebagai noise.

4. Evaluasi Hasil Clustering

Salah satu tantangan terbesar dalam unsupervised learning adalah menentukan apakah hasil clustering yang diperoleh sudah baik atau belum, karena tidak tersedia label yang bisa dijadikan acuan pasti.

Apabila ground truth atau label sebenarnya tersedia untuk keperluan validasi, dapat digunakan metrik Adjusted Rand Index (ARI), yang mengukur tingkat kesamaan antara hasil clustering dengan label yang sebenarnya. Nilai ARI sebesar 1 menunjukkan hasil yang sempurna, sedangkan nilai 0 menunjukkan hasil yang setara dengan penebakan acak.

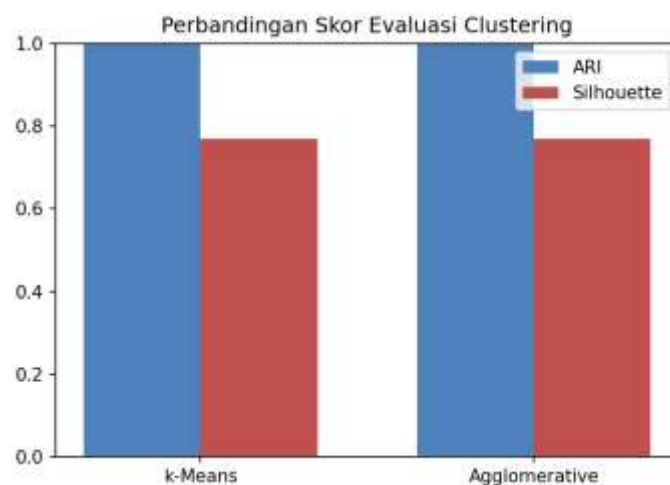
```
from sklearn.metrics import adjusted_rand_score
```

```
score = adjusted_rand_score(y, kmeans.labels_)  
print("ARI:", score)
```

Sementara itu, apabila tidak tersedia ground truth, yang merupakan situasi paling umum dijumpai pada kasus nyata, dapat digunakan silhouette score, yang menilai seberapa rapat titik-titik data dalam cluster yang sama dibandingkan dengan jaraknya terhadap cluster lain, tanpa memerlukan informasi label asli.

```
from sklearn.metrics import silhouette_score
```

```
sil = silhouette_score(X, kmeans.labels_)  
print("Silhouette score:", sil)
```



Gambar 4.1 Perbandingan skor ARI dan silhouette score untuk k-Means dan Agglomerative Clustering

Pada data `make_blobs` yang dipakai sebagai contoh, baik k-Means maupun Agglomerative Clustering menghasilkan ARI sebesar 1,0 (sama persis dengan label sebenarnya) dan silhouette score sebesar 0,77, seperti terlihat pada Gambar 4.1. Hal ini masuk akal karena data `make_blobs` memang dirancang membentuk gumpalan (blob) yang bulat dan terpisah jelas, sehingga cocok dengan asumsi kedua algoritma tersebut.

Perlu diingat bahwa nilai silhouette score yang tinggi tidak selalu berarti hasil clustering benar secara semantik, karena metrik ini hanya mengukur kekompakan secara geometris. Oleh karena itu, evaluasi hasil clustering sering perlu dikombinasikan dengan inspeksi visual dan pengetahuan domain terkait data yang digunakan.

KESIMPULAN

Berdasarkan pembahasan pada Chapter 3, dapat disimpulkan bahwa unsupervised learning terbagi menjadi dua kelompok teknik besar, yaitu transformasi data (meliputi scaling, PCA, NMF, dan t-SNE) yang berfungsi mengubah representasi data, serta clustering (meliputi k-Means, Agglomerative Clustering, dan DBSCAN) yang berfungsi mengelompokkan data berdasarkan kemiripannya.

Preprocessing dan scaling data terbukti sangat berpengaruh terhadap performa model, terutama pada algoritma yang berbasis jarak seperti SVM. Teknik reduksi dimensi seperti PCA, NMF, dan t-SNE masing-masing memiliki karakteristik dan kegunaan yang berbeda, mulai dari kompresi informasi hingga visualisasi struktur data yang kompleks. Sementara itu, tidak ada satu algoritma clustering yang paling tepat digunakan untuk semua kasus, karena pemilihannya sangat bergantung pada bentuk dan karakteristik data yang dianalisis. Terakhir, karena tidak tersedia label yang pasti pada unsupervised learning, hasil clustering perlu dievaluasi menggunakan metrik seperti Adjusted Rand Index (apabila ground truth tersedia) atau silhouette score (apabila ground truth tidak tersedia), dan tetap perlu dikonfirmasi kembali secara kualitatif.

DAFTAR PUSTAKA

Müller, A. C., & Guido, S. (2016). Introduction to Machine Learning with Python: A Guide for Data Scientists. O'Reilly Media.

Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.